Cheatsheet Programmierung 2

Daniel Appenmaier, Juni 2026

Cheatsheet

java.lang

Klasse	Methode	Statisch	Rückgabetyp
Boolean	compare(x: boolean, y: boolean)	X	int
Boolean	valueOf(s: String)	X	Boolean
Boolean	valueOf(b: boolean)	X	Boolean
Comparable<T>	compareTo(o: T)		int
Double	compare(d1: double, d2: double)	X	int
Double	valueOf(s String	X	Double
Double	valueOf(d: double)	X	Double
Integer	compare(x: int, y: int)	X	int
Integer	valueOf(s: String)	X	Integer
Integer	valueOf(i: int)	X	Integer
Long	compare(x: long, y: long)	X	int
Long	valueOf(s: String)	X	Long
Long	valueOf(l: long)	X	Long
Object	equals(object: Object)		boolean
Object	hashCode()		int
Object	toString()		String
Runnable	run()		void
String	charAt(index: int)		char
String	length()		int
String	split(regex: String)		String[]
String	toLowerCase()		String
String	toUpperCase()		String
System	currentTimeMillis()	X	long

java.time

Klasse	Methode	Statisch	Rückgabetyp
LocalDate	getDayOfMonth()		int
LocalDate	getDayOfYear()		int
LocalDate	getMonth()		Month
LocalDate	getMonthValue()		int
LocalDate	getYear()		int
LocalDate	now()	X	LocalDate
LocalDate	of(year: int, month: int, dayOfMonth)	X	LocalDate
LocalDate	parse(text: CharSequence)	X	LocalDate
LocalTime	getHour()		int
LocalTime	getMinute()		int
LocalTime	getSecond()		int
LocalTime	now()	X	LocalTime
LocalTime	of(hour: int, minute: int, second: int)	X	LocalTime
LocalTime	parse(text: CharSequence)	X	LocalTime

java.util

Klasse	Methode	Statisch	Rückgabetyp
Aufzählung	valueOf(arg0: String)	X	*Aufzählung*
Aufzählung	values()	X	*Aufzählung[]*
Collections	sort(list: List<T>)	X	void
Collections	sort(list: List<T>, c: Comparator<T>)	X	void
Comparator<T>	compare(o1: T, o2: T)		int
Entry<K, V>	getKey()		K
Entry<K, V>	getValue()		V
Entry<K, V>	setValue(value: V)		V
List<E>	add(e: E)		boolean
List<E>	add(index: int, element: E)		void
List<E>	contains(o: Object)		boolean
List<E>	forEach(action: Consumer<T>)		void
List<E>	get(index: int)		E
List<E>	of(elements: E...)	X	List<E>
List<E>	remove(index: int)		E
List<E>	remove(o: Object)		boolean
List<E>	reversed()		List<T>
List<E>	size()		int
Map<K, V>	containsKey(key: Object)		boolean
Map<K, V>	containsValue(value: Object)		boolean
Map<K, V>	entrySet()		Set<Entry<K, V>>
Map<K, V>	forEach(action: BiConsumer<K, V>)		void
Map<K, V>	get(key: Object)		V
Map<K, V>	keySet()		Set<K>
Map<K, V>	put(key: K, value: V)		V
Map<K, V>	putIfAbsent(key: K, value: V)		V
Map<K, V>	values()		Collection<V>
Optional<T>	empty()	X	Optional<T>
Optional<T>	get()		T
Optional<T>	ifPresent(action: Consumer<T>)		void
Optional<T>	ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)		void
Optional<T>	isPresent()		boolean
Optional<T>	of(t: T)	X	Optional<T>
Optional<T>	ofNullable(t: T)	X	Optional<T>
Optional<T>	orElse(other: T)		T
OptionalDouble	empty()	X	OptionalDouble
OptionalDouble	getAsDouble()		double
OptionalDouble	ifPresent(action: DoubleConsumer)		void
OptionalDouble	ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)		void
OptionalDouble	isPresent()		boolean
OptionalDouble	of(value: double)	X	OptionalDouble
OptionalDouble	orElse(other: double)		double
Random	nextInt(origin: int, bound: int)		int

java.util.function

Klasse	Methode	Statisch	Rückgabetyp
BiConsumer	accept(t: T, u: U)		void
Consumer<T>	accept(t: T)		void
DoubleConsumer	accept(value: double)		void
Function<T, R>	apply(t: T)		R
Predicate<T>	test(t: T)		boolean
ToDoubleFunction<T, R>	applyAsDouble(value: T)		double
ToIntFunction<T, R>	applyAsInt(value: T)		int

java.util.stream

Klasse	Methode	Statisch	Rückgabetyp
Collectors	groupingBy(classifier: Function<T, K>)	X	Collector<T, ?, Map<K, List<T>>>
DoubleStream	average()		OptionalDouble
DoubleStream	sum()		double
IntStream	average()		OptionalDouble
IntStream	sum()		int
Stream<T>	allMatch(predicate: Predicate<T>)		boolean
Stream<T>	anyMatch(predicate: Predicate<T>)		boolean
Stream<T>	collect(collector: Collector<T, A, R>)		R
Stream<T>	count()		long
Stream<T>	distinct()		Stream<T>
Stream<T>	filter(predicate: Predicate<T>)		Stream<T>
Stream<T>	findAny()		Optional<T>
Stream<T>	findFirst()		Optional<T>
Stream<T>	forEach(action: Consumer<T>)		void
Stream<T>	limit(maxSize: long)		Stream<T>
Stream<T>	map(mapper: Function<T, R>)		Stream<R>
Stream<T>	mapToDouble(mapper: ToDoubleFunction<T, R>)		DoubleStream
Stream<T>	mapToInt(mapper: ToIntFunction<T, R>)		IntStream
Stream<T>	max(comparator: Comparator<T>)		Optional<T>
Stream<T>	min(comparator: Comparator<T>)		Optional<T>
Stream<T>	noneMatch(predicate: Predicate<T>)		boolean
Stream<T>	skip(n: long)		Stream<T>
Stream<T>	sorted()		Stream<T>
Stream<T>	sorted(comparator: Comparator<T>)		Stream<T>
Stream<T>	toList()		List<T>

java.io

Klasse	Methode	Statisch	Rückgabetyp
PrintStream	print(obj: Object)		void
PrintStream	println()		void
PrintStream	println(x: Object)		void

org.junit.jupiter.api

Klasse	Methode	Statisch	Rückgabetyp
Assertions	assertEquals(expected: Object, actual: Object)	X	void
Assertions	assertFalse(condition: boolean)	X	void
Assertions	assertNotEquals(expected: Object, actual: Object)	X	void
Assertions	assertNotNull(actual: Object)	X	void
Assertions	assertNotSame(expected: Object, actual: Object)	X	void
Assertions	assertNull(actual: Object)	X	void
Assertions	assertSame(expected: Object, actual: Object)	X	void
Assertions	assertThrows(expectedType: Class<T>, executable: Executable)	X	T
Assertions	assertTrue(condition: boolean)	X	void
Executable	execute()		void

org.mockito und org.mockito.stubbing

Klasse	Methode	Statisch	Rückgabetyp
Mockito	when(methodCall: T)	X	OngoingStubbing<T>
MockitoAnnotations	openMocks(testClass: Object)	X	AutoCloseable
OngoingStubbing<T>	thenReturn(value: T)		OngoingStubbing<T>